CROP DISEASE PREDICTION

Deep Learning for Plant Disease Detection

Using CNN and Transfer Learning (MobileNetV2) to classify plant leaf diseases with high accuracy

Computer Vision | TensorFlow / Keras | 38 Disease Classes

Problem Statement

The challenge this project addresses

The Core Problem

01 Crop Loss at Scale

Plant diseases destroy 20–40% of global food supply annually, threatening food security in agriculture-dependent regions.

02 Manual Detection is Costly

Farmers rely on expensive expert consultations or visual inspection, which is slow, error-prone, and inaccessible in some areas.

03 Delayed Response

Without early detection, diseases spread rapidly across fields, making treatment ineffective and losses irreversible.

04 Lack of Scalable Tools

No affordable, automated, and accurate tool exists for smallholder farmers to identify diseases from a photo in real time.

Solution: A deep learning system that classifies plant diseases from leaf images with ~95% accuracy — fast, affordable, and scalable.

Project Objectives

What this project aims to achieve

1 Build an Accurate Classifier

Develop a deep learning model that classifies plant leaf images into 38 disease categories with high accuracy (target ≥90%).

2 Compare Model Approaches

Evaluate and compare a custom CNN architecture against Transfer Learning (MobileNetV2) to identify the best-performing approach.

3 Handle Real-World Data

Preprocess raw image data including augmentation, normalization, and class balance checks to ensure robust model training.

4 Evaluate Rigorously

Use accuracy, loss curves, classification report, and confusion matrix heatmap to fully assess model performance.

5 Support Deployment Readiness

Save the best model in a format ready for deployment in an application to assist farmers in real-time disease detection.

Business Understanding

Why this project matters

Problem Statement

- Plant diseases reduce crop yield and quality if not detected early
- Manual inspection is expensive and requires expert knowledge
- This system automates disease detection from leaf images
- Classifies images into 38 disease categories using deep learning

Key Stakeholders

- Farmers & Smallholders
- Agricultural Experts
- Agri-tech Companies
- Government Bodies
- Researchers

Project Roadmap

Step-by-step pipeline

01 **Business Understanding**

Define goals & stakeholders

02 **Environment Setup**

TensorFlow, Keras, libraries

03 **Data Loading**

Train / Val / Test splits

04 **Preprocessing**

Augmentation & normalization

05 **Model Building**

CNN + Transfer Learning

06 **Train & Evaluate**

Callbacks, metrics, history

07 **Final Selection**

MobileNetV2 best performer

Data Understanding

Exploring the plant disease dataset

Dataset Overview

38 Disease Classes	**3** Splits (Train/Val/Test)
128×128 Image Size (px)	**32** Batch Size

Directory Structure

```
plant_dataset/
  ├── train/
  │     └── <class_name>/
  ├── val/
  │     └── <class_name>/
  └── test/
        └── <class_name>/
```

Key Observations

Balanced Classes

Dataset is well distributed across all 38 classes — no severe skew, no oversampling needed.

No Corrupt Images

All images in train, val, and test passed PIL integrity verification.

RGB Color Mode

All images are in RGB format, consistent across all splits.

Inferred Labels

Class labels are inferred from folder names — no manual labelling required.

Environment Setup & Libraries

Imports used in the notebook

TensorFlow / Keras

Model building, training, data loading

Matplotlib

Plotting accuracy/loss curves

NumPy

Numerical operations & array handling

Scikit-learn

Classification report & confusion matrix

Pandas

Data manipulation and analysis

PIL / Pillow

Image verification and preprocessing

Load Training, Validation & Test Data

Using tf.keras.utils.image_dataset_from_directory

```
# Load training dataset
train_dataset = tf.keras.utils.image_dataset_from_directory(
    directory=f"{base_dir}/train",
    image_size=(128,128),
    batch_size=32,
    labels="inferred",
    label_mode="categorical",
    color_mode="rgb",
    shuffle=True
)
# Load validation dataset
val_dataset = tf.keras.utils.image_dataset_from_directory(
    directory=f"{base_dir}/val",
    image_size=(128,128),
    batch_size=32,
    label_mode="categorical",
    shuffle=True
)
# Load test dataset (shuffle=False for evaluation)
test_dataset = tf.keras.utils.image_dataset_from_directory(
    directory=f"{base_dir}/test",
    image_size=(128,128),
    batch_size=32,
    label_mode="categorical",
    shuffle=False
)
```

Key Parameters

`image_size`
(128, 128) px

`batch_size`
32 images/batch

`label_mode`
categorical (one-hot)

`color_mode`
rgb

`shuffle`
True for train/val

Data Augmentation

Applied to training dataset only to improve generalization

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
train_dataset = train_dataset.prefetch(buffer_size=1)
val_dataset   = val_dataset.prefetch(buffer_size=1)
test_dataset  = test_dataset.prefetch(buffer_size=1)
```

Why Augment?

- Increases effective dataset size
- Reduces overfitting
- Applied only to training set
- Val/Test remain unchanged

Random Flip

Horizontal & vertical — prevents orientation bias

Random Rotation

±10% — avoids overfitting to fixed angles

Random Zoom

±10% — adds scale variation

Prefetch

Improves GPU utilization during training

CNN Model Architecture

Custom sequential CNN with BatchNormalization

```
model = Sequential([
    data_augmentation,
    layers.Rescaling(1./225),
    Conv2D(32,(3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),
    BatchNormalization(),
    Conv2D(64,(3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),
    BatchNormalization(),
    Conv2D(128,(3,3), activation='relu', padding='same'),
    MaxPooling2D((2,2)),
    BatchNormalization(),
    GlobalAveragePooling2D(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(38, activation='softmax')
])
```

Input	128×128×3 RGB
Conv×3	32→64→128 filters
MaxPool×3	Downsample 2×2
BatchNorm	Stabilize training
GAP	Global Avg Pooling
Dense(256)	Dropout 0.5
Output	38 classes, Softmax

Training Callbacks

Smart training controls to prevent overfitting

EarlyStopping

Stops training if val_loss doesn't improve for 5 epochs. Restores best weights automatically.

```
EarlyStopping(
  monitor='val_loss',
  patience=5,
  restore_best_weights=True)
```

ModelCheckpoint

Saves the best model to disk whenever val_loss improves.

```
ModelCheckpoint(
  'cnn_model.keras',
  monitor='val_loss',
  save_best_only=True)
```

ReduceLROnPlateau

Halves learning rate if val_loss plateaus for 3 epochs. Helps escape local minima.

```
ReduceLROnPlateau(
  monitor='val_loss',
  factor=0.5,
  patience=3)
```

Model Training

Fitting the CNN model

```
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=10,
    callbacks=[early_stop, checkpoint, reduce_lr],
    verbose=1
)
model.save("cnn_model.keras")
import json
with open('train_history.json', 'w') as f:
    json.dump(history.history, f)
```

Training Configuration

Optimizer	Adam (lr=1e-4)
Loss	Categorical Crossentropy
Metric	Accuracy
Epochs	10 (with early stopping)
Batch size	32

CNN Results

Train Acc: 89.8%

Val Acc: 69.4%

Observation: Signs of overfitting

Transfer Learning — MobileNetV2

Pre-trained on ImageNet, fine-tuned for plant disease

```
base_model = MobileNetV2(
    input_shape=(128,128,3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False
inputs = tf.keras.Input(shape=(128,128,3))
x = preprocess_input(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(38, activation='softmax')(x)
model_tl = tf.keras.Model(inputs, outputs)
model_tl.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Why MobileNetV2?

- Lightweight & efficient
- Pre-trained on 1M+ ImageNet images
- Depthwise separable convolutions
- Fewer params — less overfitting
- Ideal for edge deployment

Model Evaluation & Visualization

Accuracy, loss curves & confusion matrix

```
plt.plot(history.history['accuracy'],     label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.legend(); plt.show()

plt.plot(history.history['loss'],     label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend(); plt.show()

test_loss, test_acc = model_tl.evaluate(test_dataset)
print("Test Accuracy:", test_acc)

cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)
```

Evaluation Metrics

94.4% Train Accuracy	**95.4%** Val Accuracy
0.152 Val Loss	**~95%** Test Accuracy

Confusion Matrix uses annot=True, fmt='d', cmap='Blues' for clear class-by-class visualization across all 38 disease categories.

Final Model Selection

MobileNetV2 Selected as Final Model

Metric	CNN (Custom)	MobileNetV2
Train Accuracy	89.8%	94.4%
Val Accuracy	69.4%	95.4%
Val Loss	Higher	0.152
Generalization	Weak	Strong

MobileNetV2 achieves 95.4% validation accuracy vs 69.4% for custom CNN — significantly better generalization to unseen plant disease images.